

Document

Built: 2026-07-02 10:48:14 UTC

Sections: 3

Tags: ast, compiler, concurrency, gc, goroutines, internals, memory, runtime, scheduler, ssa

Table of Contents

[1.0.0] Core Execution & Compiler Architecture

[2.0.0] The Go Runtime & Memory Management

[3.0.0] Advanced Concurrency Model & Scheduling

The Compilation Pipeline

Go's compiler transforms human-readable source code into machine executable binaries through a multi-stage pipeline. Each stage performs a specific transformation while preserving program semantics.

Lexical Analysis

The first phase converts raw source bytes into tokens. The lexer operates character by character, grouping symbols into meaningful lexical units.

```
type Token struct {  
    Type TokenType  
    Value string  
    Pos    Position  
}
```

The Internal Lexer in Go 1.26

The `cmd/compile/internal/syntax` package scans the byte buffer using a hand-written scanner. Unlike generated lexers, Go's hand-crafted approach provides better error messages and simpler debugging during compiler crashes.

Parsing into AST

Tokens are consumed by a recursive-descent parser that builds the Abstract Syntax Tree. Go 1.26 improved cycle detection for type definitions that reference themselves indirectly.

Circular type definitions can cause infinite loops in naive parsers. The Go compiler detects these during type checking, not during parsing, which means the AST is always complete.

SSA Form

The AST is lowered into Static Single Assignment form, which enables powerful optimizations like dead code elimination and constant propagation.

SSA form ensures every variable is assigned exactly once, making dataflow analysis straightforward and enabling compiler passes to reason about code without tracking mutable state across branches.

Key Optimization Passes

Pass	Description	Impact
Dead Code Elimination	Removes unreachable branches	~5-15% binary size
Inlining	Replaces calls with body	~20-40% speed
Escape Analysis	Stack vs heap allocation	Reduces GC pressure

Memory Allocator Architecture

The Go runtime uses a TCMalloc-inspired allocator. The design revolves around four core structures that work together to provide fast allocation with minimal fragmentation.

The Heap Structure

```
type mheap struct {
    lock      mutex
    spans     []*mspan
    arenas    []heapArena
    central   [numSpanClasses]mcentral
    spanalloc fixalloc
}
```

Arena-Based Allocation

Each `heapArena` manages a contiguous 64MB region of virtual memory. The `mheap` maintains a mapping from address ranges to arena metadata, enabling O(1) lookups when determining which span owns a given pointer during GC scanning.

Span Management

Spans are the fundamental allocation unit. Each span manages contiguous pages divided into equal-sized objects.

Size Class	Object Size	Page Count	Typical Use
1	8 bytes	1	bool, byte
5	48 bytes	1	small structs
17	512 bytes	1	medium objects
67	32KB	8	large allocations

Garbage Collection

Go uses a concurrent, tri-color mark-and-sweep collector with a non-generational, non-compacting design.

Understanding GC pacing is critical for latency-sensitive applications. The GC triggers when the heap grows by a configurable factor (default 100%), which can cause sudden CPU spikes if allocation bursts occur.

CPU Caches & False Sharing

Modern CPUs use cache lines (typically 64 bytes) to minimize memory latency. When multiple goroutines write to variables on the same cache line, performance degrades severely.

```
type Counter struct {
    value int64
    _     [56]byte // padding to own cache line
}
```

False sharing is invisible to profilers because no sharing actually occurs — the illusion of sharing caused by cache line granularity is the root of the performance issue.

The M:P:G Model

Go's scheduler implements the M:P:G model, where OS threads (M) execute goroutines (G) via logical processors (P).

Work-Stealing Algorithm

When a processor runs out of work, it steals goroutines from other processors' local queues. This ensures load balancing without a centralized scheduler bottleneck.

```
func stealWork(p *p) *g {
    for i := 0; i < numProcs; i++ {
        target := randomProc()
        if g := target.runq.pop(); g != nil {
            return g
        }
    }
    return nil
}
```

The Stealing Heuristic

Go uses a randomized steal policy rather than a deterministic one. Each P attempts to steal from up to 4 random Ps before falling back to the global run queue. This avoids contention on shared scheduler locks and scales linearly with core count.

The Network Poller

Network I/O in Go is non-blocking at the goroutine level. The network poller integrates with the OS event loop mechanism.

OS	Mechanism	Goroutine Parking
Linux	epoll	<code>gopark()</code>
macOS	kqueue	<code>gopark()</code>
Windows	IOCP	<code>gopark()</code>

Synchronization Primitives

Channels

Channels are the primary communication mechanism in Go, built on the `hchan` struct.

```
type hchan struct {
    qcount    uint
    dataqsiz  uint
    buf       unsafe.Pointer
    elemsize  uint16
    closed    uint32
    recvg     waitq
    sendq     waitq
}
```

Unbuffered channels synchronize both sender and receiver. A send blocks until a corresponding receive occurs, which means channel operations are also synchronization points in the memory model.

Mutex Design

Go's mutex uses a fair locking strategy with spinning for short critical sections.

Fairness in mutex design prevents goroutine starvation but adds ~15ns overhead per lock acquisition compared to a naïve spinlock. Go prioritizes fairness over raw throughput to maintain predictable scheduling behavior.